AI 220: Assignment 1

Due: October 6, 2026, at midnight (end of day)

Overview

Implement two graph search algorithms for the supplied campus map program: uniform cost search and greedy best-first search. The starter code provides the graph, map display, and interaction logic. Your task is to implement the search algorithms, returning paths that the interface can display.

Run the program from the project directory:

```
uv sync
uv run assignment-1
```

In Edit mode, click nodes to turn them on or off. In Search mode, click an enabled start node and then an enabled goal node. Hover over an edge to see its cost. The Help screen contains instructions and the current status.

Graph and search interface

The abstract base class `SearchAlgorithm` is defined in `src/assignment_1/search.py`. Implement both algorithms as subclasses of this class. Each implementation must provide:

```
def find_path(self, graph: Graph, start: Node, goal: Node) -> list[Node] | None:
```

Create separate classes named `UniformCostSearch` and `GreedyBestFirstSearch`. To run either implementation in the interface, update the import and the search instance passed to `MapApp` in `src/assignment_1/app.py`.

- `graph.neighbors(node)` returns a list of enabled neighboring `Node` objects. A disabled node has no neighbors.
- `graph.edge_cost(a, b)` returns the cost of an existing edge. Costs are Euclidean distances in the original map's pixel coordinates:

  $$c(a, b) = \sqrt{(b.x - a.x)^2 + (b.y - a.y)^2}.$$

  Use the full-precision values returned by this method, not the rounded values displayed in tooltips.
- Nodes are immutable and hashable; you can use them as dictionary keys and set members. Their coordinates are available as `node.x` and `node.y`.
- Return a list of nodes in path order, including both endpoints. Every consecutive pair must share an edge, and every node must be enabled.
- Return `None` when no path exists or either endpoint is disabled. If the start and goal are the same enabled node, return `[start]`. Do not modify the graph.

Part 1: Uniform cost search

Implement uniform cost search using `queue.PriorityQueue` from the Python standard library. Prioritize frontier nodes by their accumulated path cost from the start:

$$g(n) = \text{sum of the edge costs along the current path from the start to } n.$$

Your implementation must return a minimum-total-cost path when one exists. It must handle cycles, multiple routes to the same node, and a cheaper route discovered after a node has already been placed in the frontier. Ensure that your stopping condition preserves the minimum-cost guarantee.

Use `graph.edge_cost(current, neighbor)` to obtain each step's cost. Handle equal priorities without requiring Python to compare `Node` objects, which do not define an ordering. You may use other standard-library data structures to track costs and reconstruct paths, but do not use a library implementation of a search algorithm.

Part 2: Greedy best-first search

Implement greedy best-first search using an estimated cost-to-go as the heuristic. For this assignment, use straight-line Euclidean distance from the current node to the goal:

$$h(n) = \sqrt{(\text{goal}.x - n.x)^2 + (\text{goal}.y - n.y)^2}.$$

Compute this estimate directly from node coordinates. Do not call `graph.edge_cost(n, goal)` to compute the heuristic: a node and the goal need not share an edge.

Prioritize frontier nodes using $h(n)$ alone. Adding $g(n)$ to the priority would produce A* search, which is not the algorithm requested here. You may reuse `queue.PriorityQueue` for this part. Handle cycles and repeated nodes so the algorithm terminates on the finite graph, and reconstruct a valid path when the goal is reached. Greedy best-first search is not required to return a minimum-cost path.

Deliverables

Submit the source code for both implementations via Brightspace.